投资的收益和风险：含最低交易费的组合优化

摘　要

针对含最低交易费的资产配置问题，建立银行存款与风险资产之间的收益—风险组合优化模型。将资金总额和可承受风险作为决策参数，在给定风险上限下最大化净收益，构造供公司选择的投资方案。

将购买金额、交易费和银行存款共同纳入预算；不购买时费用为零，正购买时费用为费率乘购买金额与门槛金额的较大值。利用二元启用变量建立混合整数线性规划，并以最大单项风险损失额占初始资金比例作为风险指标。

当可用资金为 100 万元时，四资产风险上限 1% 的净收益率为 21.902%；十五资产风险上限 10% 时为 33.534%。两组各计算 61 个风险上限，代表方案与解析上界吻合。进一步把目标净收益率作为输入，最小化所需风险：四资产要求 20% 时的最小风险为 0.587599%，十五资产要求 33% 时为 9.128131%。六个可达目标均由原费用可行方案与有理数对偶下界共同证明；27% 与 41% 分别超过两组全风险范围的最大净收益率，不可达。目标均为示例，不能代表公司的实际偏好。

四资产进一步通过 81 种费用分段的枚举与连续背包计算独立核验。资金和风险参数情景揭示：最低交易费在小资金下会改变组合；将风险理解为所持资产中最大的风险率时，收益—风险曲线变成只买一项资产的阶梯，奖励集中而非分散。因此结果仅适用于所采用定义与题目参数，模型同时给出最低交易费生效条件。100 元、四资产、20% 目标的完整支持枚举反例证明：比例费用逆值只给必要下界，不能在小资金下自动当作最小风险。几何拐点保留作折中示例，实际配置由公司确认的目标与风险定义决定。

关键词：投资组合；混合整数规划；最低交易费；风险约束；独立验证

1　问题重述与数据核对

问题重述。公司有一笔数额为 M 的资金，可购买 n 种资产 Si，也可存银行（利率 r_0=5%，无费用无风险）。购买 Si 的平均收益率为 r_i、风险损失率为 q_i，交易费率为 p_i，购买额不超过 u_i 时按 u_i 计费；总体风险按所投资资产中最大的一个风险度量。原题要求：（1）对表 1 的四种资产设计投资组合，使净收益尽可能大、总体风险尽可能小；（2）讨论一般情形，并对表 10 的十五种资产计算。原始数据来自全国组委会官方历史赛题汇编第 11 页 [1]，收益率、风险损失率和费率在计算时由百分数转为小数，门槛金额以元计。本文的处理是：先固定风险上限求收益最优方案，再反解公司给定目标净收益时的最低所需风险；未获得偏好时保留方案族，拐点仅作几何折中示例。

资金总额 M 与公司风险偏好未提供。本文取 M=1,000,000 元作为数值分析基准，并考察 M=100、1,000、10,000 元。资金总额保留为模型参数，各风险上限对应一个收益最优方案；公司可根据可承受风险选择配置。

文献定位。含固定交易费与最小交易量的组合选择，在文献中通常写成混合整数线性规划，规模大时再用启发式求解 [4]；本题收益与损失率为给定参数、没有历史序列，不能估计 Markowitz 均值—方差模型所需的协方差 [5]。本文采用的最大单项损失额度量，与以最坏损失而非方差衡量风险、并可用线性规划处理的极小极大思路同属一类 [6]，但对象不同：它约束各资产风险损失额的最大值，而非历史收益序列的最小值。

表 1　四资产题设参数

资产	收益率%	风险率%	费率%	门槛元
S1	28	2.5	1	103
S2	21	1.5	2	198
S3	23	5.5	4.5	52
S4	25	2.6	6.5	40

要求与回答位置。四资产在给定风险上限下的最优方案见第 4 节；十五资产见第 5 节；含最低交易费的一般模型与精确求解见第 3 节；最优性证据与一般资金规模下的条件见第 6 节；资金、参数和风险定义的敏感性见第 7 节；目标收益对应的最低风险、缺少偏好时的折中示例及决策边界见第 8 节。

2　假设、定义与符号

采用单时期、无卖空、资金可连续分割；题目收益率与风险率视为本期给定参数。银行利率 r_0=5%，无风险和交易费。没有观测时间序列，不估计相关性、收益分布或市场预测精度，也不将风险率解释成违约概率。税收、流动性与跨期再平衡不在题目数据内。

下文采用表 2 的符号和单位；购买本金、交易费用与银行余额分别记账，以便核对预算和净收益。

表 2　符号与单位

符号	含义与单位
M	初始可用资金，元
x_0 / x_i	银行存款 / 第 i 项购买本金，元
r_i / q_i / p_i	收益率、题设风险损失率、交易费率，无量纲
u_i / f_i	费用计费门槛 / 实际交易费，元
y_i	启用变量，0 或 1
ρ / R	风险上限 / 最大单项风险暴露额占 M 的比例
T	公司要求的最低净收益率，本文只用示例输入
λ / A(λ)	非负预算对偶参数 / 风险界系数，无量纲

风险采用最大单项金额暴露占初始资金的比例，而非所选资产风险率的最大值。这是本文为体现分散投资效果所采用的操作定义，不是由题目数据识别出的损失分布。原文的另一种解释在第 7 节比较。u_i 是最低计费基数，不是最低购买额；交易费函数为：

$$f_i(0) = 0, \qquad f_i(x_i) = p_i \max(x_i, u_i) \quad (x_i > 0) \tag{1}$$

单时期、连续分割和忽略税收使预算模型可线性化；若交易要求整手购买，则需增加整数购买单位，本文结果不再直接适用。若费用另有资金支付或原题收益率已扣费，也须改变下面的现金流模型。

3　一般模型与精确求解

3.1　现金流与风险约束

交易费在购买时从初始资金支付。期末财富由银行本息及风险资产本息组成；期末财富减去初始投入，得到净利润。因此费用只计入利润一次，不能因为预算也含费用就再重复扣减。

$$W = (1 + r_0)x_0 + \sum_{i=1}^{n} (1 + r_i)x_i, \qquad P = W - M \tag{2}$$

$$P = r_0 x_0 + \sum_{i=1}^{n} r_i x_i - \sum_{i=1}^{n} f_i \tag{3}$$

$$x_0 + \sum_{i=1}^{n} (x_i + f_i) = M, \qquad x_0, x_i \geq 0 \tag{4}$$

$$R(\mathbf{x}) = \max_{1 \le i \le n} \frac{q_i x_i}{M} \le \rho \tag{5}$$

收益率为净利润除以初始资金。风险指标仅约束资金暴露；没有收益与损失的联合分布，故不把风险参数额外扣入期望收益。给定不同风险上限得到一族条件最优方案，全银行提供净收益率 5% 的可行基线。

3.2 混合整数线性表示

将金额除以初始资金归一化，避免资金尺度使约束系数悬殊。对每种资产加入二元启用变量；其具体定义和优化问题如下（所有逐项约束均对 i=1,⋯,n 成立）：

$$v_i = \frac{x_i}{M}, \quad b = \frac{x_0}{M}, \quad h_i = \frac{f_i}{M}, \quad t_i = \frac{u_i}{M} \tag{6}$$

$$\max \quad r_0 b + \sum_{i=1}^{n} (r_i v_i - h_i) \tag{7}$$

$$b + \sum_{i=1}^{n} (v_i + h_i) = 1, \qquad b, v_i, h_i \ge 0 \tag{8}$$

$$0 \le v_i \le y_i, \quad h_i \ge p_i v_i, \quad h_i \ge p_i t_i y_i \tag{9}$$

$$h_i \le p_i \max(1, t_i) y_i, \quad q_i v_i \le \rho, \quad y_i \in \{0, 1\} \tag{10}$$

当 y_i=0 时购买额与费用均为零；当 y_i=1 时费用至少等于两种计费值的较大者。由于过高费用既直接减少利润又占用可生息预算，最优解不会多付费用，故在最优解处该表示与分段收费等价。启用变量为 1 而购买额为零时，若费用为正，取消启用并把费用转存银行会严格增加收益；零费率时取消启用不影响收益。故总能选择与分段收费一致的最优解。费用上界利用归一化购买额不超过 1 推导，避免任意超大的常数。

用 SciPy 的 milp 接口 [2] 和 HiGHS 求解 [3]，相对最优间隙设为 1e-9；每次求解均得到最优状态。每组扫描 61 个风险上限，四资产范围 0 到 3%，十五资产范围 0 到 60%。离散扫描展示收益包络，不声称给出全部连续前沿或所有并列最优解。

4 四资产结果

四资产的代表方案见表 3，相应购买额和费用列于表 4；两表共同给出风险上限、实际暴露与现金流的对应关系。

图 1 将代表方案放回收益包络，显示提高风险上限的边际收益逐渐减小。风险上限 0.5% 时四种资产都有配置，并保留存款；到 1% 时方案集中在 S1 与 S2。该结果说明 “分散越多越好” 不是收益最优的通用规则；具体组合取决于风险限制和净收益机会成本。不同风险上限不能直接作优劣排名。

表 3　四资产代表方案收益

风险上限	实际风险	净收益率	净收益元	银行元
0.500%	0.500%	17.764%	177,638.05	158,192.31
1.000%	1.000%	21.902%	219,019.61	0.00
2.000%	2.000%	25.176%	251,764.71	0.00

表 4　四资产购买本金及交易费用（元）

风险上限	S1 本金	S2 本金	S3 本金	S4 本金	费用合计
0.500%	200,000.00	333,333.33	90,909.09	192,307.69	25,257.58
1.000%	400,000.00	584,313.73	0.00	0.00	15,686.27
2.000%	800,000.00	188,235.29	0.00	0.00	11,764.71

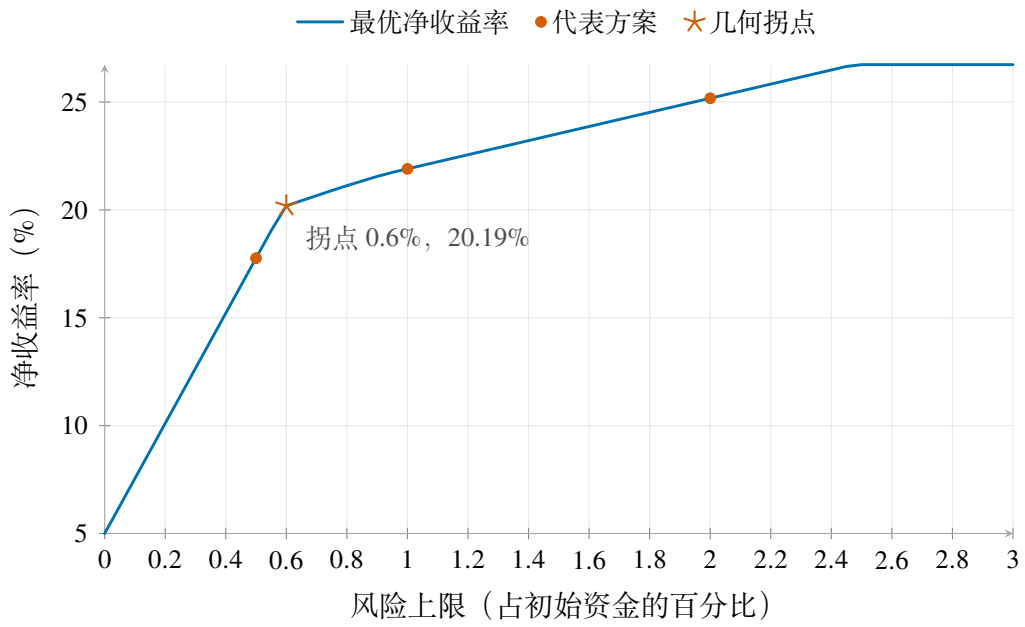


图 1　四资产风险上限与最优净收益率

注：圆点为代表方案，星号为几何拐点（第 8 节），横轴为约束上限。

5　十五资产结果与一般扩展

十五资产的代表结果见表 5；比较风险上限与实际风险，可以区分仍受风险约束的方案和收益已进入平台的方案。

表 5　十五资产代表方案收益

风险上限	实际风险	净收益率	净收益元
5.000%	5.000%	26.433%	264,333.07
10.000%	10.000%	33.534%	335,341.21
20.000%	20.000%	35.862%	358,621.68

十五资产采用与四资产完全相同的模型，全部参数见支撑材料 data/assets15.csv。一般 n 情形有 3n+1 个变量，其中 n 个二元变量；新增资产不要求更换方法。规模更大时求解耗时与启用组合有关，应保留实际最优间隙，不能只凭求解器返回可行解宣称全局最优。

风险上限为 10% 时的启用资产及交易费用见表 6，可据此逐项核对该代表方案的购买支出。

表 6　十五资产代表配置

风险上限 10% 的启用资产	购买本金元	交易费元
S3	166,666.67	10,000.00
S7	147,058.82	8,235.29
S9	182,598.19	4,930.15
S10	250,000.00	7,250.00
S13	217,391.30	5,869.57

图 2 显示十五资产收益包络的平台；继续放宽上限不一定增加收益。该方案交易费合计 36,285.01 元。无风险限制时，四资产最优净收益率 26.733%，十五资产为 40.943%；相应集中投资的风险更高。没有公司偏好输入，本文不把某个风险上限当作唯一最优；第 8 节按明确规则给出一个几何拐点。

6　独立核验与最优性证据

首先从原始 CSV 独立重算预算、分段费用、利润与实际风险，检查非负性和约束；随后检查零风险退化为全银行、收益包络单调以及无风险限制下的解析答案。

四资产的核验不调用优化求解器：逐项取“不买 / 低于门槛 / 高于门槛”三种状态，共 81 种（四项资产各 3 种状态）。在每个状态中，费用已是仿射函数，资金约束成为带上下限的连续背包；先分配下限，再按边际利润与边际预算比从高到低分配，获得该状态的确切最优值。取所有状态的最大值，与 MILP 比较。

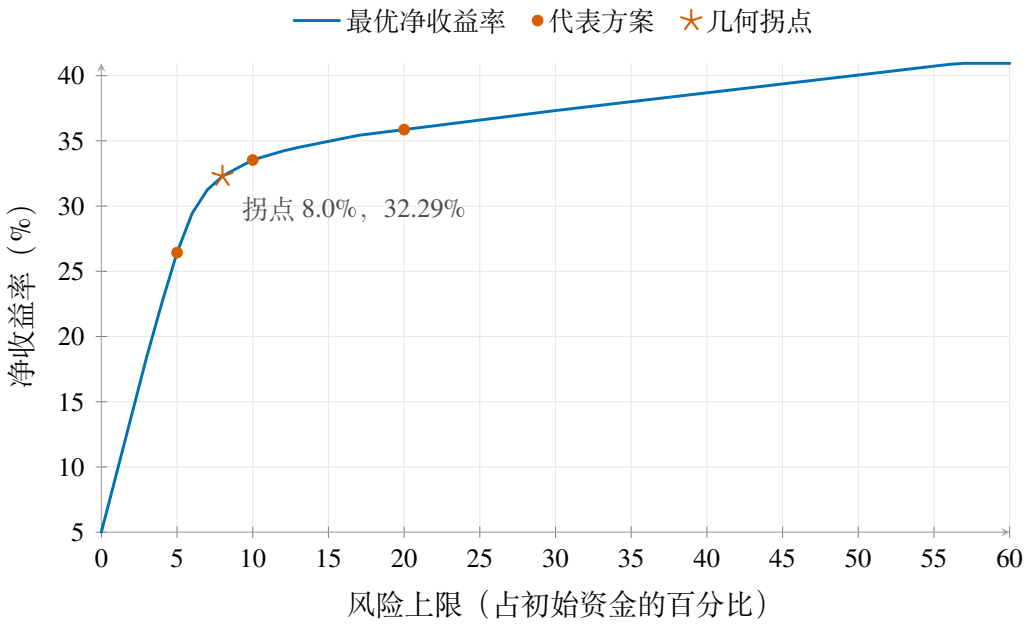


图 2　十五资产最优收益包络

注：圆点为代表方案，星号为几何拐点（第 8 节）；平台区不表示风险上限全部被用尽。

6.1　比例费用松弛与解析上界

命题 1（上界）。去掉最低计费门槛、仅保留比例费用 $p_i \cdot x_i$ 的松弛问题，其最优值 U 不小于原问题的最优值。

证明. 设 x 为原问题的任一可行方案，其交易费 $f_i{=}p_i \cdot \max(x_i, u_i) \geq p_i \cdot x_i$。把 x 放入松弛问题，各项费用降为 $p_i \cdot x_i$，节省的费用可存入银行；预算仍满足，风险暴露不变，净收益不减。因此 x 在松弛问题中可行且收益不低，U 是原问题的上界，不是另一个经验近似答案。

□

将银行余额由预算消去，松弛问题为：

$$w_i = 1 + p_i, \qquad g_i = r_i - r_0 - (1 + r_0)p_i \tag{11}$$

$$\max \quad r_0 + \sum_{i=1}^{n} g_i v_i \tag{12}$$

$$\sum_{i=1}^{n} w_i v_i \leq 1, \qquad 0 \leq v_i \leq \frac{\rho}{q_i} \tag{13}$$

其中 w_i 是每单位购买本金消耗的预算，g_i 是扣除银行机会成本后的增量净收益。命题 2（松弛问题的解；设各 $q_i{>}0$，η_i 并列时任取一种次序，结果相同）。只有 $g_i{>}0$ 的资产值得购买；按单位预算增益 $\eta_i{=}g_i/w_i$ 从高到低依次分配，每种资产取剩余预算 B 与单项风险上限所允许的较小值：

$$\eta_i = \frac{g_i}{w_i}, \qquad v_i = \min\left(\frac{B}{w_i}, \frac{\rho}{q_i}\right) \tag{14}$$

证明（交换论证）. 设某最优分配中，增益比较低的资产获得了正的预算，而增益比较高的资产尚未到达风险上限。把一小部分预算从前者转给后者，预算消耗不变，目标函数增加（两者增益比之差乘转移量），与最优性矛盾。故最优分配必按 η_i 排序，依次填满上限或耗尽预算，即得精确解 U。若净增益都非正，保留银行存款即可。

□

命题 3（最优性证书）。由命题 1，原问题任一可行方案的收益率不超过 U，因此可行方案的收益率等于 U 时它是全局最优：

$$\frac{P_{\text{feasible}}}{M} \le \frac{P^*}{M} \le U(\rho) \tag{15}$$

本次两组各三个代表方案以及第 8.2 节的两个几何拐点方案均达到上界。小资金方案可能落在最低费用段，不能自动使用上界相等的证书。

6.2 一般资金规模下的解析条件

命题 4（充分资金阈值）。记比例费用松弛的最优配置为归一化购买额 v*，给定风险上限，定义：

$$M_{\text{crit}} = \max_{i:v_i^*>0} \frac{u_i}{v_i^*}, \qquad M \ge M_{\text{crit}} \tag{16}$$

证明. 当 M ≥ Mcrit 时，每个启用资产的购买本金 $M \cdot v_i^*$ 不低于门槛 u_i，其费用 $p_i \cdot \max(x_i, u_i)$ 等于 $p_i \cdot x_i$，故松弛最优配置对原问题可行且收益等于 U；由命题 1 它是原问题最优解，净收益率与 M 无关。

□

若没有资产启用，阈值记为零。若松弛最优配置唯一，且启用资产费率均为正，这个条件也是必要的：当 M<Mcrit 时，达到 U 的可行方案只能是 v* 本身，而 v* 中某个启用资产的购买额低于其门槛，实际费用高于 $p_i \cdot x_i$，收益严格小于 U。本题两组启用资产的费率均为正，故这两个阈值是恰好的临界资金；松弛最优不唯一时，Mcrit 依赖于所取的最优配置，仍只是充分条件。低于阈值时应用分段费用模型求解。

四资产风险上限 1% 时，资金阈值约为 338.86 元；十五资产风险上限 10% 时约为 2601.34 元。为避免金额四舍五入低于边界，可分别取 339 元与 2,602 元作为充分资金条件。该结论解释了第 7 节中资金超过门槛后最优收益率保持不变的现象。

6.3 数值证据与检验边界

核验共 12 项，全部通过，记录了实际误差、约束残差及证书间隙；这 12 项包含可行性、会计恒等式和性质检验，并非 12 种独立求解算法；真正独立的最优性核验

为费用分段枚举与比例费用解析上界。资金为 100 万元时命题 4 说明整数模型与比例费用松弛等价，“达到解析上界”的证书在主结果上主要检验求解器；检验最低计费建模的是小资金情景，其费用分段枚举只覆盖四资产，十五资产的小资金情景依靠求解器的最优间隙证书。数值误差以 1e-7 的无量纲容差处理，金额表按分四舍五入，极小负银行残差在展示时记为零。

7　资金规模、参数与解释敏感性

四资产，风险上限 1%

资金变化的结果分别列于表 7 和表 8。净收益率随小资金规模变化，反映最低交易费的影响；进入比例计费区间后，归一化收益趋于稳定。

表 7　四资产资金规模情景

资金元	净收益率	交易费元
100.00	18.426%	5.97
1,000.00	21.902%	15.69
10,000.00	21.902%	156.86
1,000,000.00	21.902%	15,686.27

十五资产，风险上限 10%

表 8　十五资产资金规模情景

资金元	净收益率	交易费元
100.00	6.082%	14.01
1,000.00	30.388%	56.93
10,000.00	33.534%	362.85
1,000,000.00	33.534%	36,285.01

资金情景是自行设定的压力检查，100 元情景偏离原题“相当大资金”的描述，专用于检验门槛。资金大到所有启用投资超过门槛后，比例归一化使最优收益率不再随 M 改变；资金小时固定最低计费会影响收益和选择。

四资产：将全部 q_i 乘以 0.9、1、1.1，在同一风险上限下重新求解，收益率依次为 22.266%、21.902%、21.591%。

十五资产：将全部 q_i 乘以 0.9、1、1.1，在同一风险上限下重新求解，收益率依次为 33.917%、33.534%、32.977%。

这三个乘数没有经验分布依据，只是参数情景，不是置信区间或真实发生概率。风险率增大使可行域缩小，最优收益不应增大，本次检查符合该性质。

若将总体风险定义为已选资产中最大的风险率 q_i，即每个被选资产自身的 q_i 都不得超过上限，这个上限本身是一个风险率，要在它自己的尺度上扫描，不能沿用按金额占比定义的 1% 与 10%。结果是阶梯形的单资产方案：四资产，上限达到 1.5% 时只买 S2（净收益率 18.627%）；上限达到 2.5% 时只买 S1（净收益率 26.733%）；十五资产，上限达到 23.0% 时只买 S15（净收益率 6.877%）；上限达到 33.4% 时只买 S8（净收益率 27.255%）；上限达到 40.0% 时只买 S10（净收益率 32.945%）；上限达到 60.0% 时只买 S3（净收益率 40.943%）。该读法只要求被选资产各自不超限，不惩罚把全部资金押在一项资产上，奖励集中而不是分散，与题面 “投资越分散，总的风险越小” 的说法相反，所以本文不采用，风险指标的操作定义应与公司的决策目标一致。

8 从目标到投资决策

8.1 目标净收益对应的最低风险

公司若给出最低净收益率 T，直接在第 3 节原预算与最低收费约束下，加入净利润不低于 TM 的条件，并最小化最大单项损失比例 R。这把决策条件变成可检验的约束，不需要从曲线形状推断公司的偏好。银行提供 5% 净收益，因此 T 不超过 5% 时最小风险为零。

$$\min R \quad \text{s.t.} \quad P \geq TM, \quad x_0 + \sum_i (x_i + f_i) = M, \quad q_i x_i \leq RM. \tag{17}$$

命题 5（目标风险下界）。沿用第 6 节 w_i、g_i，令 zi=x_i/M。任取非负 λ，并记 A(λ) 为下面的风险界系数，原费用方案均满足：

$$A(\lambda) = \sum_i \frac{(g_i - \lambda w_i)_+}{q_i}, \qquad P/M \leq r_0 + \lambda + RA(\lambda). \tag{18}$$

$$A(\lambda) > 0 \quad \Longrightarrow \quad R \geq \frac{T - r_0 - \lambda}{A(\lambda)}. \tag{19}$$

证明. 实际费用不低于比例费用，所以原方案的 zi 满足购买预算加比例费不超过总资金，即 $w_i \cdot$ zi 的总和不超过 1；净收益率不超过 r_0 加 $g_i \cdot$ zi 的总和。把 g_i 拆成 $\lambda \cdot w_i$ 与剩余项，负剩余项删去，正剩余项用 zi≤R/q_i 代入，即得上界。目标收益不低于 T，移项得到风险必要下界。

□

按第 6 节单位预算增益的次序，比例费用上包络是连续、非减的分段直线。取边际资产的 η_i 作为 λ（全部正增益资产均到上限时取零），就能反解目标所在分支。四资产包络共 5 段、十五资产共 13 段；完整公式、端点与原模型证书由独立有理数核验器重算。若反解配置的所有正费率启用资产均超过计费门槛，该可行解达到风险

下界，便是原收费模型的全局最小风险。资金条件按该目标的实际配置计算，不能沿用另一风险上限的门槛。

$$M_{\mathrm{crit}}(T) = \max_{i:z_i^*>0,\, p_i>0} \frac{u_i}{z_i^*}, \qquad M \geq M_{\mathrm{crit}}(T). \tag{20}$$

表 9 将目标净收益率转换为最低所需风险。所有目标均为示例输入；原手续费可行证书与对偶下界精确一致。

表 9　目标收益与最小风险

资产数	目标净收益率	最小风险	充分资金元
4	20.000%	0.587599%	505.45
4	22.000%	1.029940%	345.88
4	25.000%	1.946108%	944.75
15	30.000%	6.225538%	13,285.56
15	33.000%	9.128131%	2,813.29
15	38.000%	34.979825%	734.14

表中的充分资金由未舍入有理门槛向上取整到分，满足该金额即不会因门槛舍入而失去可行性。它不是将原最优购买额逐项取整到分；实际下单取整后仍须重验。

例如四资产 20% 目标对应的风险为 12870/2190269，净利润恰为 20 万元；十五资产 33% 目标对应 7406225814/81136277107，净利润恰为 33 万元。两组最大净收益率分别为 27/101≈26.732673% 与 217/530≈40.943396%；预算全部用于含手续费的最高净收益资产可达到这两个上界，所以 27% 与 41% 目标确实不可达。证书采用连续金额的有理数；按分取整后须重验预算、收益与风险。

小资金反例。四资产 M=100 元、T=20%，比例费逆值仍为 0.587599%。但该风险上限下购买额均低于计费门槛；完整枚举 16 个支持集，原费用最大净收益仅为 13.224127%，故这一风险及以下都达不到目标。另有只买 S1、风险 1.747989% 的原费用方案恰达到 20%，说明目标可达而松弛界不紧；这里没有把这个可行风险称为精确最小值。数值 MILP 可继续求小资金的原问题，界与最优性结论必须分开。

8.2　缺少偏好时的几何折中

优点是费用与预算一致、两组问题共用一般模型、有可复现输出与独立最优性证据。限制是最大单项损失只是一种题设风险代理，不是组合方差、尾部总损失或概率保证；缺少相关性与收益分布，不能代表真实市场投资。返回的是条件最优方案之一，而非所有可能的并列方案。

折中示例。没有风险偏好输入时，保留收益—风险方案族；下面的拐点只是归一化几何折中示例：把净收益率从零风险值到饱和值、风险上限从零到饱和上限分别重标为 [0,1]，取曲线高出两端连线最多的一点。四资产拐点在风险上限 0.6%：购买

S1 240,000.00 元、S2 400,000.00 元、S3 109,090.91 元、S4 221,220.66 元，银行存款为零，净收益率 20.191%；已取得可得增益的 69.9%，风险上限只用了饱和上限（2.5%）的 24%。拐点前后局部斜率约为 23.0 与 4.7，即风险上限每提高 0.1 个百分点，净收益率约增加 2.30 与 0.47 个百分点。

十五资产的拐点在 8%（扫描步长 1 个百分点）：S3 133,333.33 元、S7 117,647.06 元、S8 189,986.75 元、S9 150,093.81 元、S10 200,000.00 元、S13 173,913.04 元；净收益率 32.290%，取得增益的 75.9%，风险上限仅为饱和上限（57%）的 14%。该曲线较平缓，拐点前后斜率只有 1.05 与 0.63，几何折中的依据弱于四资产。拐点取自上述网格，斜率也是网格上的割线；把网格细到 0.1 个百分点后，十五资产的拐点在 7.7%（净收益率 32.10%），四资产仍在 0.6%（20.19%），净收益率相差不到 0.2 个百分点，所以拐点只作近似位置使用。两个拐点处的可行方案都等于解析上界（20.191%、32.290%），因而是各自风险上限下的全局最优。拐点规则只是归一化后的几何折中约定，不代表公司的风险偏好，公司偏好明确时应直接按其风险上限选择。

8.3 执行误差、比较基线与适用范围

拐点示例的执行误差与数据误差。拐点方案中有资产达到风险上限；在风险率不变时，增持这些资产可能越界，减持则不会因该项而越界。预算调整和其他资产的变化也会影响最终风险，不能把所有方向的误差都判为越界。对各项投资额独立加入 ±5% 的随机误差，各做 200 次：四资产只有 10% 的试验仍在上限内，风险额的中位数超出上限 2.2%；十五资产为 4%，超出 3.1%。误差放大到 ±10%，超出幅度约翻倍（4.0%、6.2%）。这不是净收益的问题，而是约束被穿透：带误差的购买额合计常超过资金（±5% 时四资产有 50% 的试验、十五资产有 59%），这里按预算等比例缩减后再计算，净收益率的中位数仍是 20.119% 与 32.242%（±5%），仅比无误差低 0.07 与 0.05 个百分点。以上比例来自 200 次试验，按二项分布估计，95% 区间的半宽在比例为一半时约为 7 个百分点，不宜解读到个位数。

预留安全余量的代价（图 3）。把风险上限收紧 5% 再求最优，四资产净收益率从 20.191% 降到 19.551%，十五资产从 32.290% 降到 32.031%；收紧 10% 则降到 18.785% 与 31.506%。在风险率与总资金不变、执行误差仅改变购买额的条件下，设预留比例为 s、逐项正向误差不超过 e。执行后的最大单项风险不超过原上限的 $(1-s)(1+e)$ 倍；只要该乘积不超过 1，就能保证风险约束，必要的预算缩减不会增加风险。该保证不覆盖风险率本身的误差。

参数误差。对收益率、风险率、费率各项独立加入 ±10% 的随机误差（门槛不变），各做 200 次重新求解：最优资产组合几乎不变（四资产 100%，十五资产 99.5% 的试验中选择的资产相同），重新优化的净收益率在 18.921%–21.382%（四资产）与 30.854%–33.562%（十五资产，均为 5%–95% 分位）之间。原购买额在费率扰动后可能超出预算。保持原比例的收益对照先按新风险上限、再按含最低费用的预算缩减；未调整购买额只作诊断。若沿用原方案不调整，风险额中位数超出上限 6.5%、7.3%。

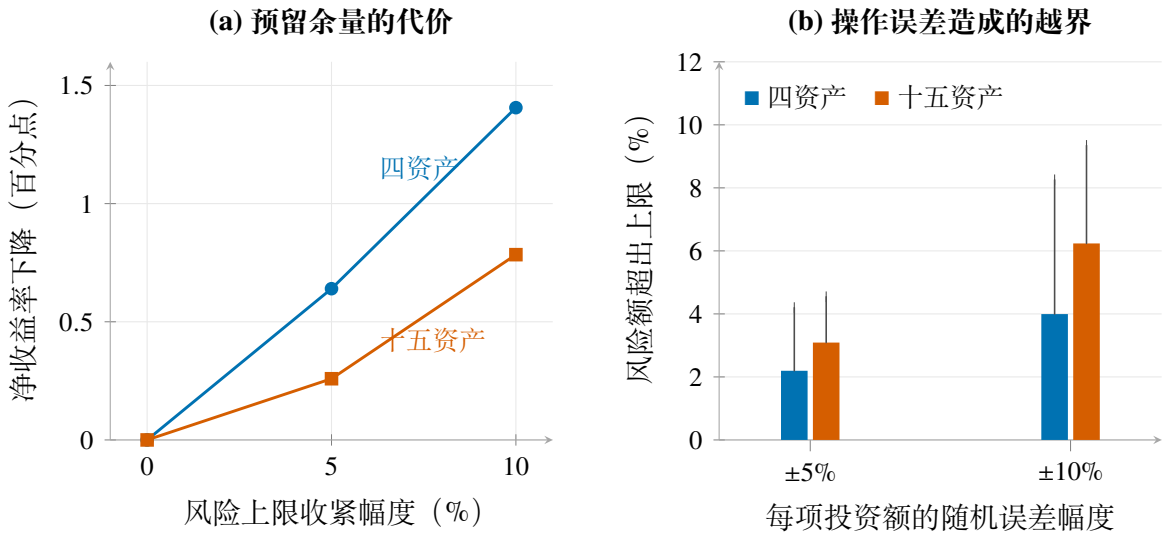


图 3　安全余量的代价与操作误差的危害

注：左图：相对未预留余量的方案，收紧风险上限后最优净收益率的下降；右图：拐点方案在每项投资额独立加随机误差后风险额超出上限的中位数（柱）与 95% 分位（须线上端），各 200 次，种子固定；左图三点之间的连线只是引导视线，未计算中间值。

结论：该拐点示例的组成比较稳健，风险上限的遵守不稳健；这些误差范围同样是设定的情景，没有经验分布依据。

与简单规则比较（图 4）。在 100 万元下，按命题 2 的单位预算增益 η_i 从高到低、每项买到风险上限允许的最大额的贪心规则，在三个风险上限（拐点的一半、拐点、两倍）下，两组共六个情形都与混合整数规划的最优值一致（差距小于 1e-12），这是命题 2 与命题 4 的直接结果：资金足够大时最低计费门槛不起作用，问题等价于带比例费用的线性规划。按“收益率减费率”排序在这两组数据上给出相同的次序，所以结果也相同，但它不是 η_i，一般数据下不保证最优。按“净收益率除以风险率”排序的规则在两倍拐点上限下低 2.4（四资产）与 4.3（十五资产）个百分点；在可买资产中平均分配的规则在拐点上限下只有 11.796% 与 21.960%，仅存银行为 5%。所以最低计费的整数模型在大资金下没有额外价值，在小资金（第 7 节）下才需要；决定结果的是“用满每项允许额”的排序。

逐项剔除（图 5）。在拐点上限下去掉任一被选资产重新求解，四资产净收益率降到 14.757%–18.868%（原为 20.191%），十五资产降到 29.887%–31.319%（原为 32.290%）。在给定资产集合与风险上限下，各被选资产均提高最优收益，最容易替代的是十五资产中的 S9（去掉后少 0.97 个百分点）。

另一种读法：在费用门槛与预算条件经核对后，利用柯西—施瓦茨不等式求精确配置。把风险率当作标准差，且各资产收益相互独立（这是本文为该读法补充的假设，题目没有给出），总风险取各项标准差的平方和开方。同样把上限设为拐点的 0.6% 与 8%，最优净收益率只有 13.367% 与 17.405%，十五资产的方案分散到 12 项资产。反过来，题设“最大单项损失”下的拐点方案，按这种读法的风险是 1.19% 与

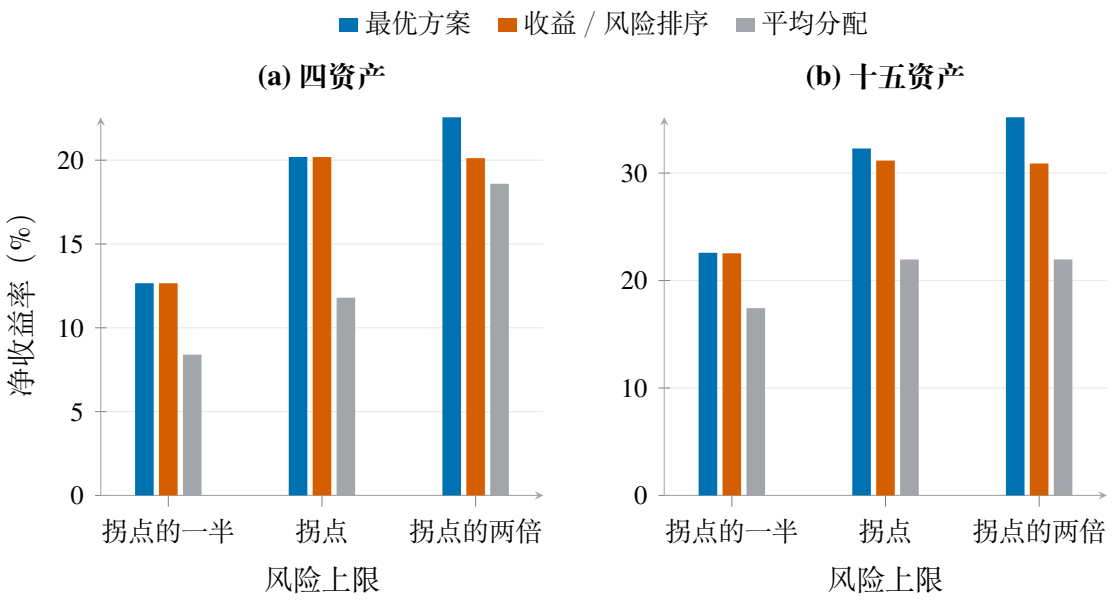


图 4　简单规则与最优方案的差距

注：三个风险上限为拐点的一半、拐点和两倍；按 η_i 排序的贪心规则（以及在这两组数据上次序相同的收益率减费率排序）在六个情形都与最优一致，所以并入第一类柱。

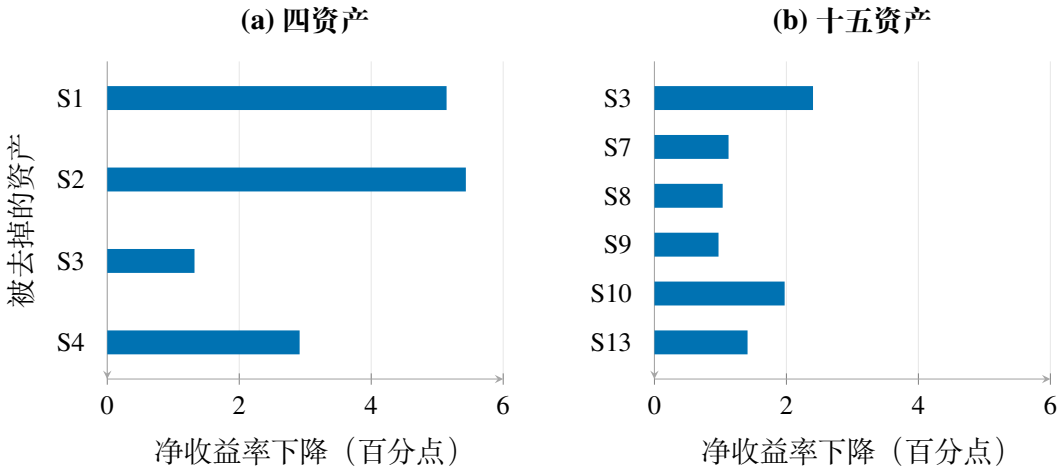


图 5　逐项剔除资产的收益损失

注：在拐点风险上限下，每次去掉一个被选资产后重新求解；以原方案收益率为基准，两图使用相同尺度。损失表示给定资产集合和风险上限下的替代代价。

19.0%，远高于 0.6% 与 8%。两种风险定义给出的方案不同，须先与公司确认；本段只作对照。

建议公司先确认可用资金、目标净收益率和风险定义，再由目标反解最低风险或按明确风险上限选择方案；若有预计的操作误差，可按上述误差界计算必要余量（5%–10% 仅为本文示例）；若真实目标改成总损失、流动性或多时期决策，需要重新建模。提高模型复杂度不能弥补缺失数据。

AI 工具使用声明

本论文由 AI 助手（OpenAI Codex；后期修订使用 Anthropic Claude Sonnet 5.5，通过 Claude Code）完成选题、模型推导、编程、计算、结果解释与论文撰写，2026 年 10 月 12 日由 OpenAI Codex 增加目标收益逆问题、独立有理证书及小资金反例；没有记录人工逐项审核；各项工具、过程与核验情况见支撑材料《AI 工具使用详情》。

参考文献

[1] 全国大学生数学建模竞赛组委会. 1992—2000 年全国大学生数学建模竞赛赛题汇编：1998 年 A 题，PDF 第 11 页 [EB/OL]. [2026-10-07]. https://www.mcm.edu.cn/upload_cn/node/1/SkAh1A7Q6f2dd01e58aa621f920e79f45cd5d255.pdf

[2] SCIPY DEVELOPERS. scipy.optimize.milp: SciPy API reference[EB/OL]. [2026-10-07]. https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html

[3] HiGHS DEVELOPMENT TEAM. HiGHS: high performance software for linear optimization[EB/OL]. [2026-10-07]. https://highs.dev/

[4] KELLERER H, MANSINI R, SPERANZA M G. Selecting portfolios with fixed costs and minimum transaction lots[J]. Annals of Operations Research, 2000, 99: 287-304.

[5] MARKOWITZ H. Portfolio selection[J]. The Journal of Finance, 1952, 7(1): 77-91.

[6] YOUNG M R. A minimax portfolio selection rule with linear programming solution[J]. Management Science, 1998, 44(5): 673-683.

附录 A 数据与支撑材料文件列表

十五资产参数来自原题第二张表 [1]，供不运行程序的读者复核；收益、风险与费用列单位均为百分数，门槛为元。

附录表 10 列全十五资产输入，使正文结果可以从原始收益率、风险率、费用和门槛逐项复核。

表 10　十五资产题设参数

资产	收益率%	风险率%	费率%	门槛元
S1	9.6	42	2.1	181
S2	18.5	54	3.2	407
S3	49.4	60	6	428
S4	23.9	42	1.5	549
S5	8.1	1.2	7.6	270
S6	14	39	3.4	397
S7	40.7	68	5.6	178
S8	31.2	33.4	3.1	220
S9	33.6	53.3	2.7	475
S10	36.8	40	2.9	248
S11	11.8	31	5.1	195
S12	9	5.5	5.7	320
S13	35	46	2.7	267
S14	9.4	5.3	4.5	328
S15	15	23	7.6	131

supporting_materials.zip 含以下文件；reproduced/ 为重跑时新生成，不纳入原提交包。各文件对应本文的数据、数值结果与计算过程。

AI 工具使用详情.pdf
README.md
build_paper.py
check_alternatives.py
check_capital_threshold.py
check_recommendation.py
check_return_target.py
check_robustness.py
check_target_certificates.py
code/model.py
code/validate.py
data/assets15.csv
data/assets4.csv
equations.json
figures/fifteen-frontier.png
figures/four-frontier.png
paper/ai-use.tex
paper/main.tex
reference/alternatives.json
reference/checks.json
reference/fifteen-frontier.csv
reference/four-frontier.csv
reference/recommendation.json
reference/results.json
reference/return-targets.json
reference/robustness.json
reference/target-certificates.json
reference/target-checks.json
requirements.txt
run_demo.py
runtime-versions.json
sources.json

附录 B 完整源程序

以下列出建模、独立核验和各节数字所依赖的全部源程序（共九个文件）；运行方法见附录 A 所列的运行说明。论文排版用的脚本不产生任何结果，不在此列。

code/model.py

```
"""Original solution to a historical CUMCM problem; parameters are demo scenarios."""
import argparse
import csv
import json
from pathlib import Path
import numpy as np
from scipy.optimize import milp, Bounds, LinearConstraint
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt


def load(name):
    rows=list(csv.DictReader(open(Path('raw/data')/name,encoding='utf-8')))
    return rows, np.array([[float(d[k]) for k in
    ['return_pct','risk_pct','fee_pct','threshold_yuan']] for d in rows])


def solve(data, budget, cap, risk_scale=1.0, interpretation='amount'):
    n=len(data);r,q,p,u=data.T.copy();r/=100;q=q/100*risk_scale;p/=100;u/=budget
    # v = [bank, investments, fees, activation binaries], all money scaled by budget.
    size=1+3*n;ix=np.arange(1,1+n);jf=ix+n;ky=jf+n
    c=np.r_[-.05,-r,np.ones(n),np.zeros(n)]
    upper=np.r_[1,np.ones(n),p*np.maximum(1,u),np.ones(n)]
    if interpretation=='rate':
        upper[ix[q>cap+1e-12]]=0
    rows=[];lb=[];ub=[]
    a=np.zeros(size);a[0]=1;a[ix]=1;a[jf]=1
    rows.append(a);lb.append(1);ub.append(1)
    for i in range(n):
        for coeffs,hi in [({int(ix[i]):p[i],int(jf[i]):-1},0),
                          ({int(ky[i]):p[i]*u[i],int(jf[i]):-1},0),
                          ({int(ix[i]):1,int(ky[i]):-1},0),
                          ({int(jf[i]):1,int(ky[i]):-upper[jf[i]]},0)]:
            a=np.zeros(size)
            for j,value in coeffs.items():a[j]=value
            rows.append(a);lb.append(-np.inf);ub.append(hi)
        if interpretation=='amount':
            a=np.zeros(size);a[ix[i]]=q[i]
            rows.append(a);lb.append(-np.inf);ub.append(cap)
    res=milp(c,integrality=np.r_[np.zeros(1+2*n),np.ones(n)],
             bounds=Bounds(np.zeros(size),upper),
             constraints=LinearConstraint(np.array(rows),np.array(lb),np.array(ub)),
             options={'mip_rel_gap':1e-9,'time_limit':20})
    if not res.success:raise RuntimeError(res.message)
    active=res.x[ky]>.5  # the solver's binaries decide which assets are bought; a residual x on
    an inactive asset is rounding noise
    x=np.where(active,np.maximum(res.x[ix],0),0)*budget
    fees=np.where(active,p*np.maximum(x,data[:,3]),0)
```

```
    bank=budget-x.sum()-fees.sum()
    if bank<0:
        assert bank>-1e-6*budget,'budget exceeded beyond solver tolerance'
        bank=0.0
    profit=.05*bank+np.dot(r,x)-fees.sum()
    risk=float(np.max(q*x)/budget) if interpretation=='amount' else
    float(max(q[x>1e-7],default=0))
    return {'budget_yuan':budget,'risk_cap':float(cap),'risk_scale':risk_scale,
            'interpretation':interpretation,'investments_yuan':x.tolist(),
            'fees_yuan':fees.tolist(),'bank_yuan':float(bank),
            'profit_yuan':float(profit),'net_return':float(profit/budget),
            'actual_risk':risk,'mip_gap':float(res.mip_gap),
            'solver_message':res.message}


def main():
    parser=argparse.ArgumentParser();parser.add_argument('--output',type=Path,required=True)
    out=parser.parse_args().output
    results={'historical_demo':True,'bank_rate':.05,'primary_budget_yuan':1e6,
             'risk_definition':'max_i(q_i*x_i)/M; x_i is purchased principal; fees paid from M',
             'groups':{},'sensitivity':{},'alternative_interpretation':{}}
    for key,name,end,caps in [('four','assets4.csv',.03,[.005,.01,.02]),
                              ('fifteen','assets15.csv',.6,[.05,.1,.2])]:
        rows,data=load(name)
        frontier=[solve(data,1e6,cap) for cap in np.linspace(0,end,61)]
        selected=[solve(data,1e6,cap) for cap in caps]
        results['groups'][key]={'assets':rows,'frontier':frontier,'selected':selected,
            'unrestricted':solve(data,1e6,1.0)}
        results['sensitivity'][key]={'budget':[solve(data,m,caps[1]) for m in
        [100,1000,10000,1e6]],
            'risk_stress':[solve(data,1e6,caps[1],scale) for scale in [.9,1,1.1]]}
        results['alternative_interpretation'][key]=solve(data,1e6,caps[1],interpretation='rate')
        with open(out/f'{key}-frontier.csv','w',newline='') as f:
            w=csv.writer(f);w.writerow(['risk_cap','actual_risk','net_return','profit_yuan','bank_
            yuan'])
            w.writerows([d[k] for k in
            ['risk_cap','actual_risk','net_return','profit_yuan','bank_yuan']] for d in frontier)
        fig,ax=plt.subplots(figsize=(6.8,3.8))
        ax.plot([100*d['risk_cap'] for d in frontier],[100*d['net_return'] for d in
        frontier],color='#245B61',lw=2)
        ax.scatter([100*d['actual_risk'] for d in selected],[100*d['net_return'] for d in
        selected],color='#BA6238',zorder=3)
        ax.set(xlabel='Risk cap (% of initial budget)',ylabel='Net return (%)',title=f'{len(rows)}
        assets | budget = 1,000,000 yuan')
        ax.grid(alpha=.2);fig.tight_layout();fig.savefig(out/f'{key}-frontier.png',dpi=180);plt.cl
        ose(fig)
    (out/'results.json').write_text(json.dumps(results,ensure_ascii=False,indent=2,allow_nan=False
    ))


if __name__=='__main__':main()
```

code/validate.py

```
"""Independent arithmetic, fee-regime enumeration and analytical upper bounds."""
import argparse
```

```
import csv
import itertools
import json
from pathlib import Path
import numpy as np


def load(name):
    with open(Path('raw/data')/name,encoding='utf-8') as f:
        rows=list(csv.DictReader(f))
    return np.array([[float(d[k]) for k in ['return_pct','risk_pct','fee_pct','threshold_yuan']]
    for d in rows])


def regime_optimum(data, budget, cap):
    # off / minimum-fee region / proportional-fee region: solve each box knapsack by ratios.
    r,q,p,u=data.T.copy();r/=100;q/=100;p/=100;u/=budget
    best=.05
    for regimes in itertools.product(range(3),repeat=len(data)):
        lo=[];hi=[];weight=[];gain=[];fixed=0.
        for i,state in enumerate(regimes):
            risk_upper=cap/q[i]
            if state==0:a=b=0.;w=1.;g=0.
            elif state==1:
                a=0.;b=min(u[i],risk_upper);w=1.;g=r[i]-.05
                fixed+=p[i]*u[i]
            else:
                a=u[i];b=risk_upper;w=1+p[i];g=r[i]-.05-1.05*p[i]
            lo.append(a);hi.append(b);weight.append(w);gain.append(g)
        if any(b<a-1e-12 for a,b in zip(lo,hi)):continue
        remaining=1-fixed-sum(w*a for w,a in zip(weight,lo))
        if remaining < -1e-12:continue
        value=.05-1.05*fixed+sum(g*a for g,a in zip(gain,lo))
        for i in sorted(range(len(data)),key=lambda j:gain[j]/weight[j],reverse=True):
            if gain[i]<=0:continue
            extra=min(max(remaining,0)/weight[i],hi[i]-lo[i])
            value+=gain[i]*extra;remaining-=weight[i]*extra
        best=max(best,value)
    return best


def relaxed_upper(data,cap,scale=1.):
    # Ignore minimum fees. Nonnegative bank is a budget slack, so fractional knapsack is exact.
    r,q,p,_=data.T.copy();r/=100;q=q/100*scale;p/=100
    w=1+p;g=r-.05-1.05*p;left=1.;value=.05
    for i in sorted(range(len(data)),key=lambda j:g[j]/w[j],reverse=True):
        if g[i]<=0:continue
        xi=min(left/w[i],cap/q[i]);value+=g[i]*xi;left-=w[i]*xi
    return value


def main():
    a=argparse.ArgumentParser();a.add_argument('--results',type=Path,required=True);a.add_argument
    ('--output',type=Path,required=True)
    args=a.parse_args();result=json.loads(args.results.read_text());checks=[]
```

```
def check(name,passed,evidence):checks.append({'name':name,'passed':bool(passed),'evidence':evidence})
for key,file in [('four','assets4.csv'),('fifteen','assets15.csv')]:
    data=load(file);group=result['groups'][key]
    samples=group['frontier']+group['selected']+[group['unrestricted']]+result['sensitivity'][key]['budget']+result['sensitivity'][key]['risk_stress']+[result['alternative_interpretation'][key]]
    residuals=[];errors=[];violation=[]
    for d in samples:
        M=d['budget_yuan'];x=np.array(d['investments_yuan']);r,q,p,u=data.T.copy();r/=100;q=q/100*d['risk_scale'];p/=100
        fee=np.where(x>1e-7,p*np.maximum(x,u),0)
        profit=.05*d['bank_yuan']+np.dot(r,x)-fee.sum()
        risk=max(q[x>1e-7],default=0) if d['interpretation']=='rate' else max(q*x)/M
        residuals.append(abs(d['bank_yuan']+x.sum()+fee.sum()-M)/M)
        errors.extend([abs(profit-d['profit_yuan'])/M,np.max(abs(fee-d['fees_yuan']))/M,abs(risk-d['actual_risk'])])
        violation.append(max(0,-min(x)/M,-d['bank_yuan']/M,risk-d['risk_cap']))
    check(key+' feasibility and accounting',max(residuals+errors+violation)<1e-7,
          f'{len(samples)} portfolios; budget residual={max(residuals):.3g}, accounting error={max(errors):.3g}, violation={max(violation):.3g}')
    front=group['frontier'];check(key+' frontier and bank boundary',
        abs(front[0]['net_return']-.05)<1e-9 and all(b['net_return']>=a['net_return']-1e-8 for a,b in zip(front,front[1:])),
        f'zero risk gives {front[0]["net_return"]:.9f}; {len(front)} caps checked for nondecreasing optimal return')
    unrestricted=max([.05]+[(ri/100-pi/100)/(1+pi/100) for ri,_,pi,_ in data])
    check(key+' unrestricted analytical answer',abs(unrestricted-group['unrestricted']['net_return'])<1e-8,
          f'best single-asset proportional return={unrestricted:.12f}; fees above thresholds, computed={group["unrestricted"]["net_return"]:.12f}')
    selected=group['selected'];gaps=[relaxed_upper(data,d['risk_cap'])-d['net_return'] for d in selected]
    check(key+' analytical optimality certificate',max(abs(g) for g in gaps)<1e-7,
          f'Ignoring minimum fees gives a valid upper bound; selected-point gaps={gaps}')
    if key=='four':
        exact=front+selected+[group['unrestricted']]+result['sensitivity'][key]['budget']
        gaps=[abs(regime_optimum(data,d['budget_yuan'],d['risk_cap'])-d['net_return']) for d in exact]
        check('four exhaustive fee-regime optimum',max(gaps)<1e-7,
              f'3^4=81 fee regimes per portfolio; {len(exact)} cases; max return gap={max(gaps):.3g}; no optimization solver used')
    stress=result['sensitivity'][key]['risk_stress']
    check(key+' risk stress direction',all(b['net_return']<=a['net_return']+1e-8 for a,b in zip(stress,stress[1:])),
          'risk multiplier 0.9/1.0/1.1 returns='+str([d['net_return'] for d in stress]))
# The supplied asset table has no stochastic observations; test a fee discontinuity explicitly.
p=.02;u=198
check('fee discontinuity and threshold semantics',p*max(.001,u)==3.96 and p*max(198,u)==3.96,
      'S2: investment=0 has fee=0; investment=0.001 has fee=3.96; investment=198 has fee=3.96. u is not a minimum purchase quantity.')
args.output.write_text(json.dumps(checks,ensure_ascii=False,indent=2))
print(json.dumps(checks,ensure_ascii=False,indent=2))
raise SystemExit(0 if all(d['passed'] for d in checks) else 1)
```

```
if __name__=='__main__':main()
```

run_demo.py

```
"""Run the anonymous support package from a clean work directory."""
import json
from pathlib import Path
import shutil
import subprocess
import sys
import tempfile

root=Path(__file__).resolve().parent
with tempfile.TemporaryDirectory(prefix='model-demo-') as temp:
    work=Path(temp);(work/'raw/data').mkdir(parents=True)
    for path in (root/'data').glob('*.csv'):
        shutil.copyfile(path,work/'raw/data'/path.name)
    out=work/'output';out.mkdir()
    subprocess.run([sys.executable,str(root/'code/model.py'),'--output',str(out)],cwd=work,check=T
    rue)
    subprocess.run([sys.executable,str(root/'code/validate.py'),'--results',str(out/'results.json'
    ),
                    '--output',str(work/'checks.json')],cwd=work,check=True)
    actual=json.loads((out/'results.json').read_text())
    expected=json.loads((root/'reference/results.json').read_text())
    for group in ['four','fifteen']:
        for a,b in zip(actual['groups'][group]['selected'],expected['groups'][group]['selected']):
            assert abs(a['net_return']-b['net_return'])<1e-7
    target=root/'reproduced'
    if target.exists():raise FileExistsError('Refusing to replace reproduced/')
    shutil.copytree(out,target);shutil.copyfile(work/'checks.json',target/'checks.json')
    print('Independent checks passed; selected report results reproduced within 1e-7.')
```

check_capital_threshold.py

```
import importlib.util,json
from pathlib import Path
import numpy as np
base=Path(__file__).resolve().parent;r=json.loads((base/'reference/results.json').read_text())
spec=importlib.util.spec_from_file_location('model',base/'code/model.py');m=importlib.util.module_
from_spec(spec);spec.loader.exec_module(m)
(base/'reproduced').mkdir(exist_ok=True)
checks=[]
for key in ['four','fifteen']:
 group=r['groups'][key];d=group['selected'][1]
 data=np.array([[float(a[k]) for k in ['return_pct','risk_pct','fee_pct','threshold_yuan']] for a
 in group['assets']])
 critical=max(float(a['threshold_yuan'])/(x/d['budget_yuan']) for a,x in
 zip(group['assets'],d['investments_yuan']) if x>1e-7)
 for budget in [critical,np.ceil(critical)]:
  got=m.solve(data,budget,d['risk_cap'])
  gap=abs(got['net_return']-d['net_return'])
  assert gap<1e-7
```

```
  checks.append({'group':key,'sufficient_threshold_yuan':critical,'budget_yuan':float(budget),'net_return_gap':gap,'passed':True})
(base/'reproduced/capital-threshold-check.json').write_text(json.dumps(checks,indent=2))
print(checks)
```

check_recommendation.py

```
"""Locate the knee of each recorded risk-return frontier and certify it with the analytic upper bound."""
import csv,importlib.util,json
from pathlib import Path
import numpy as np
base=Path(__file__).resolve().parent;ref=base/'reference';r=json.loads((ref/'results.json').read_text())
def load(name):
 spec=importlib.util.spec_from_file_location(name,base/f'code/{name}.py');m=importlib.util.module_from_spec(spec);spec.loader.exec_module(m);return
 m
model,validate=load('model'),load('validate')
out={'rule':'knee = point of the saturating frontier farthest above the chord from zero risk to the smallest risk that reaches the maximum return; both axes rescaled to [0,1]','groups':{}}
for key in ['four','fifteen']:
 rows=list(csv.DictReader((ref/f'{key}-frontier.csv').open()))
 x=np.array([float(a['risk_cap']) for a in rows]);y=np.array([float(a['net_return']) for a in rows])
 sat=x[np.argmax(y>=y.max()-1e-9)];keep=x<=sat
 gap=(y[keep]-y[0])/(y.max()-y[0])-x[keep]/sat;i=int(gap.argmax())
 group=r['groups'][key];data=np.array([[float(a[k]) for k in ['return_pct','risk_pct','fee_pct','threshold_yuan']] for a in group['assets']])
 got=model.solve(data,1e6,float(x[i]));bound=validate.relaxed_upper(data,float(x[i]))
 assert abs(got['net_return']-y[i])<1e-7 and abs(got['net_return']-bound)<1e-7
 out['groups'][key]={'grid_step':float(x[1]-x[0]),'saturation_risk':float(sat),'max_net_return':float(y.max()),'knee_risk':float(x[i]),'knee_net_return':float(y[i]),
  'share_of_gain':float((y[i]-y[0])/(y.max()-y[0])),'share_of_saturation_risk':float(x[i]/sat),
  'slope_before':float((y[i]-y[i-1])/(x[i]-x[i-1])),'slope_after':float((y[i+1]-y[i])/(x[i+1]-x[i])),
  'analytic_upper_bound':float(bound),'investments_yuan':got['investments_yuan'],'fees_yuan':got['fees_yuan'],'bank_yuan':got['bank_yuan'],'net_profit_yuan':got['profit_yuan']}
(base/'reproduced').mkdir(exist_ok=True);(base/'reproduced/recommendation.json').write_text(json.dumps(out,indent=2))
print(json.dumps({k:{a:b for a,b in v.items() if not isinstance(b,list)} for k,v in out['groups'].items()},indent=1))
```

check_robustness.py

```
"""Execution error, safety margin price and independent parameter errors around each recommended knee portfolio."""
import csv,importlib.util,json
from pathlib import Path
import numpy as np
base=Path(__file__).resolve().parent;ref=base/'reference'
spec=importlib.util.spec_from_file_location('model',base/'code/model.py');model=importlib.util.module_from_spec(spec);spec.loader.exec_module(model)
rec=json.loads((ref/'recommendation.json').read_text())
rng=np.random.default_rng(20261007);M=1e6;N=200
```

```
def load(name):
    return np.array([[float(d[k]) for k in ['return_pct','risk_pct','fee_pct','threshold_yuan']] for d in csv.DictReader((base/'data'/name).open())])
def net(data,x):
    r,q,p,u=data.T.copy();r/=100;p/=100
    fees=np.where(x>1e-7,p*np.maximum(x,u),0);return float((.05*(M-x.sum()-fees.sum())+r@x-fees.sum())/M)
def risk(data,x):return float(np.max(data[:,1]/100*x)/M)
def spent(data,x):
    r,q,p,u=data.T.copy();p/=100
    return float(x.sum()+np.where(x>1e-7,p*np.maximum(x,u),0).sum())
def within_budget(data,x):
    """A noisy execution cannot spend more than M: scale the purchases down (bisection, fees depend on the amounts) until it fits."""
    if spent(data,x)<=M*(1+1e-12):return x,False
    lo,hi=0.,1.
    for _ in range(80):
        mid=(lo+hi)/2
        if spent(data,x*mid)<=M:lo=mid
        else:hi=mid
    return x*lo,True
out={'seed':20261007,'draws':N,'groups':{}}
for key,name in [('four','assets4.csv'),('fifteen','assets15.csv')]:
    data=load(name);g=rec['groups'][key];cap=g['knee_risk'];x0=np.array(g['investments_yuan']);nom=model.solve(data,M,cap)
    res={'cap':cap,'nominal_net_return':nom['net_return'],'nominal_risk':risk(data,x0)}
    ex={}
    for e in (.05,.10):
        rs=[];nr=[];cut=0
        for _ in range(N):
            x,scaled=within_budget(data,x0*(1+rng.uniform(-e,e,len(x0))));cut+=scaled
            rs.append(risk(data,x)/cap-1);nr.append(net(data,x))
        rs=np.array(rs);ex[f'{e:.2f}']={'share_within_cap':float((rs<=1e-9).mean()),'median_overshoot':float(np.median(rs)),'p95_overshoot':float(np.quantile(rs,.95)),'median_net_return':float(np.median(nr)),'share_scaled_to_budget':cut/N}
    res['execution_error']=ex
    res['buffer_price']=[{'buffer':b,'cap':cap*(1-b),'net_return':model.solve(data,M,cap*(1-b))['net_return']} for b in (0,.05,.10)]
    dr=[];same=[];over=[];short=[];overspent=[];risk_scaled=[];budget_scaled=[];executed_spent=[];executed_risk=[]
    sel=set(np.flatnonzero(x0>1e-6))
    for _ in range(N):
        d=data*(1+rng.uniform(-.10,.10,data.shape));d[:,3]=data[:,3]
        s=model.solve(d,M,cap);dr.append(s['net_return'])
        same.append(set(np.flatnonzero(np.array(s['investments_yuan'])>1e-6))==sel)
        over.append(risk(d,x0)/cap-1);overspent.append(max(0.,spent(d,x0)-M))
        # Keep the original proportions, but do not compare an infeasible purchase.
        factor=min(1.,cap/risk(d,x0)) if risk(d,x0)>0 else 1.
        kept,scaled=within_budget(d,x0*factor)
        risk_scaled.append(factor<1.);budget_scaled.append(scaled)
        executed_spent.append(spent(d,kept));executed_risk.append(risk(d,kept))
        assert executed_spent[-1]<=M*(1+1e-12) and executed_risk[-1]<=cap+1e-12
        short.append(s['net_return']-net(d,kept))
    res['parameter_error_10pct']={'reoptimized_net_return':{'median':float(np.median(dr)),'p05':float(np.quantile(dr,.05)),'p95':float(np.quantile(dr,.95))},
```

```
        'share_same_assets':float(np.mean(same)),'kept_plan_risk_overshoot':{'median':float(np.median(over)),'p95':float(np.quantile(over,.95))},
        'kept_plan_return_shortfall':{'median':float(np.median(short)),'p95':float(np.quantile(short,.95))},
        'kept_plan_execution':{'rule':'Keep proportions; reduce to perturbed risk cap, then reduce to budget with actual minimum fees. Unadjusted risk overshoot is a diagnostic, not an executed plan.',
            'raw_over_budget_count':int(np.count_nonzero(np.array(overspent)>1e-6)),'raw_max_overspend_yuan':float(max(overspent)),
            'share_risk_reduced':float(np.mean(risk_scaled)),'share_budget_reduced_after_risk':float(np.mean(budget_scaled)),
            'maximum_spent_yuan':float(max(executed_spent)),'maximum_risk':float(max(executed_risk)),
            'minimum_return_shortfall':float(min(short))}}
    out['groups'][key]=res
(base/'reproduced').mkdir(exist_ok=True);(base/'reproduced/robustness.json').write_text(json.dumps(out,indent=2));print(json.dumps(out,indent=1))
```

check_alternatives.py

```
"""Baselines, asset-by-asset substitution and an independent-risk reading, around each recommended knee portfolio."""
import csv,importlib.util,json
from pathlib import Path
import numpy as np
base=Path(__file__).resolve().parent;ref=base/'reference'
spec=importlib.util.spec_from_file_location('model',base/'code/model.py');model=importlib.util.module_from_spec(spec);spec.loader.exec_module(model)
rec=json.loads((ref/'recommendation.json').read_text());M=1e6
def load(name):
    return np.array([[float(d[k]) for k in ['return_pct','risk_pct','fee_pct','threshold_yuan']] for d in csv.DictReader((base/'data'/name).open())])
def net(data,x):
    r,q,p,u=data.T.copy();r/=100;p/=100
    fees=np.where(x>1e-7,p*np.maximum(x,u),0);return float((.05*(M-x.sum()-fees.sum())+r@x-fees.sum())/M)
def greedy(data,cap,key):
    # buy assets in order of key, each up to the cap-implied amount, while the budget (with fees) allows
    r,q,p,u=data.T.copy();r/=100;q/=100;p/=100
    x=np.zeros(len(r));left=M
    for i in np.argsort(-key(r,q,p)):
        if r[i]-p[i]<=.05*(1+p[i]):continue
        amt=min(cap*M/q[i],left/(1+p[i]))
        if amt>=u[i]:x[i]=amt;left-=amt*(1+p[i])
    return x
def equal_weight(data,cap):
    r,q,p,u=data.T.copy();r/=100;q/=100;p/=100
    ok=[i for i in range(len(r)) if r[i]-p[i]>.05*(1+p[i])]
    if not ok:return np.zeros(len(r))
    t=min(cap*M/max(q[i] for i in ok),M/(sum(1+p[i] for i in ok)))
    x=np.zeros(len(r));x[ok]=t;return x
def std_risk(data,cap):
    r,q,p,u=data.T.copy();r/=100;q/=100;p/=100;n=len(r);g=(r-p-.05*(1+p))
    # Cauchy-Schwarz gives the exact relaxed optimum when cash is slack and
    # every active purchase is above its fee threshold. Check those premises.
```

```
    v=np.maximum(g,0)/q;length=np.linalg.norm(v)
    z=np.zeros(n) if length==0 else cap*v/(q*length)
    x=z*M;active=x>1e-7
    fees=np.where(active,p*np.maximum(x,u),0)
    if (x[active]<u[active]).any() or x.sum()+fees.sum()>M+1e-6:
        raise ValueError('Analytic independent-risk allocation is outside its fee/budget regime')
    return x

out={'groups':{}}
for key,name in [('four','assets4.csv'),('fifteen','assets15.csv')]:
    data=load(name);g=rec['groups'][key];cap=g['knee_risk'];x0=np.array(g['investments_yuan']);q=d
    ata[:,1]/100
    caps=[cap/2,cap,2*cap];res={'cap':cap,'baselines':[]}
    for c in caps:
        opt=model.solve(data,M,c)['net_return']
        res['baselines'].append({'cap':c,'optimal':opt,'bank_only':.05,
            'greedy_return_over_risk':net(data,greedy(data,c,lambda
            r,q,p:(r-p)/np.maximum(q,1e-9))),
            'greedy_net_return':net(data,greedy(data,c,lambda r,q,p:r-p)),
            'greedy_eta':net(data,greedy(data,c,lambda r,q,p:(r-p)/(1+p))),
            'equal_weight_eligible':net(data,equal_weight(data,c))})
    chosen=[int(i) for i in np.flatnonzero(x0>1e-6)];nom=model.solve(data,M,cap)['net_return']
    res['drop_one']=[{'asset':i+1,'net_return_without':model.solve(np.delete(data,i,axis=0),M,cap)
    ['net_return']} for i in
    chosen]
    z=std_risk(data,cap);res['independent_risk_reading']={'cap':cap,'net_return':net(data,z),
        'knee_plan_std_risk':float(np.sqrt(np.sum((q*x0)**2))/M),'std_plan_max_item_risk':float(np
        .max(q*z)/M),
        'overlap_assets':len(set(np.flatnonzero(z>1))&set(chosen)),'std_plan_assets':int((z>1).sum
        ()),'knee_plan_assets':len(chosen)}
    # the other reading of "risk": every held asset must have loss rate q_i <= cap (a cap on a
    rate, not on an amount), swept on its own scale
    steps=[];last=None
    for c in sorted({0.005,*(q/1.0).tolist(),*((q+1e-6).tolist()),0.7}):
        sol=model.solve(data,M,float(c),interpretation='rate');held=[i+1 for i,v in
        enumerate(sol['investments_yuan']) if v>1e-6]
        k=(round(sol['net_return'],5),tuple(held))
        if k!=last:steps.append({'cap':float(c),'net_return':sol['net_return'],'held_assets':held}
        );last=k
    res['rate_reading_frontier']=steps
    res['nominal']=nom;out['groups'][key]=res
(base/'reproduced').mkdir(exist_ok=True);(base/'reproduced/alternatives.json').write_text(json.dum
ps(out,indent=2));print(json.dumps(out,indent=1))
```

check_return_target.py

```
"""Find the least single-asset risk exposure for a supplied net-return target.

Targets are illustrative preferences, not inferred company or market data.
The original minimum-fee budget and profit definitions are retained.
"""
import argparse
import csv
import json
from pathlib import Path
```

```
import numpy as np
from scipy.optimize import Bounds, LinearConstraint, milp


def solve_target(data, budget, target):
    data = np.asarray(data, dtype=float)
    if (data.ndim != 2 or data.shape[1] != 4 or not len(data)
            or not np.isfinite(data).all() or (data[:, 1] <= 0).any()
            or (data[:, 2:] < 0).any() or not np.isfinite(budget)
            or budget <= 0 or not np.isfinite(target)):
        raise ValueError('Finite asset rows, positive risk/budget and nonnegative fees required')
    n = len(data)
    r, q, p, u = data.T.copy()
    r /= 100
    q /= 100
    p /= 100
    u /= budget
    upper_return = float(max(.05, np.max((r - p) / (1 + p))))
    result = {'budget_yuan': float(budget), 'target_net_return': float(target),
              'proportional_fee_return_upper_bound': upper_return}
    if target > upper_return:
        return {**result, 'status': 'infeasible',
                'evidence': 'target exceeds unrestricted proportional-fee upper bound'}
    if target <= .05:
        return {**result, 'status': 'optimal', 'minimum_risk': 0.,
                'investments_yuan': [0.] * n, 'fees_yuan': [0.] * n,
                'bank_yuan': float(budget), 'net_return': .05,
                'evidence': 'bank-only feasible portfolio and nonnegative-risk lower bound'}
    # Variables: bank, principals, fees, activation binaries, risk ceiling.
    size = 2 + 3 * n
    ix = np.arange(1, 1 + n)
    jf, ky, iz = ix + n, ix + 2 * n, size - 1
    upper = np.r_[1., np.ones(n), p * np.maximum(1, u), np.ones(n), max(q)]
    rows, lower, higher = [], [], []

    def constraint(coefficients, lo=-np.inf, hi=0.):
        row = np.zeros(size)
        for index, value in coefficients.items():
            row[index] = value
        rows.append(row)
        lower.append(lo)
        higher.append(hi)

    constraint({0: 1., **{int(i): 1. for i in np.r_[ix, jf]}}, lo=1., hi=1.)
    constraint({0: .05, **{int(i): float(a) for i, a in zip(ix, r)},
                **{int(i): -1. for i in jf}}, lo=target, hi=np.inf)
    for i in range(n):
        x, f, y = int(ix[i]), int(jf[i]), int(ky[i])
        constraint({x: p[i], f: -1.})
        constraint({y: p[i] * u[i], f: -1.})
        constraint({x: 1., y: -1.})
        constraint({f: 1., y: -upper[f]})
        constraint({x: q[i], iz: -1.})
    objective = np.zeros(size)
    objective[iz] = 1.
    fit = milp(objective, integrality=np.r_[np.zeros(1 + 2 * n), np.ones(n), 0.],
```

```
                bounds=Bounds(np.zeros(size), upper),
                constraints=LinearConstraint(np.asarray(rows), lower, higher),
                options={'mip_rel_gap': 1e-9, 'time_limit': 20})
    if fit.status == 2:
        return {**result, 'status': 'infeasible', 'evidence': 'MILP infeasibility status',
                'solver_message': fit.message}
    if not fit.success:
        raise RuntimeError(fit.message)
    x = np.where(fit.x[ky] > .5, np.maximum(fit.x[ix], 0.), 0.) * budget
    # Epigraph fees may exceed the actual fee under a risk-minimizing objective.
    # Recompute actual fees and put every saved yuan into the bank. This raises
    # return without changing risk, so the feasible-target formulation is exact.
    fees = np.where(x > 0, p * np.maximum(x, data[:, 3]), 0.)
    bank = budget - x.sum() - fees.sum()
    profit = .05 * bank + r @ x - fees.sum()
    risk = float(np.max(q * x) / budget)
    if bank < -budget * 1e-7 or profit / budget < target - 1e-7:
        raise RuntimeError('Reconstructed original-fee portfolio is infeasible')
    return {**result, 'status': 'optimal', 'minimum_risk': risk,
            'investments_yuan': x.tolist(), 'fees_yuan': fees.tolist(),
            'bank_yuan': float(bank), 'net_return': float(profit / budget),
            'solver_risk': float(fit.x[iz]), 'mip_gap': float(fit.mip_gap),
            'evidence': 'numeric MILP optimum; exact independent bound checked separately'}


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--budget', type=float, default=1_000_000.)
    parser.add_argument('--group', choices=['four', 'fifteen'])
    parser.add_argument('--target', type=float, action='append',
                        help='Net return as a fraction, e.g. 0.20; requires --group')
    parser.add_argument('--output', type=Path)
    args = parser.parse_args()
    if args.target and not args.group:
        parser.error('--target requires --group')
    root = Path(__file__).resolve().parent
    target = args.output or root / 'reproduced/return-targets.json'
    if target.exists():
        raise FileExistsError('Refusing to replace an existing target result')
    scenarios = {'four': ('assets4.csv', [.05, .20, .22, .25, .27]),
                 'fifteen': ('assets15.csv', [.05, .30, .33, .38, .41])}
    result = {'preference_source': 'illustrative inputs, not supplied by the company',
              'risk_definition': 'max_i(q_i*x_i)/initial_budget', 'groups': {}}
    for key, (filename, defaults) in scenarios.items():
        if args.group and key != args.group:
            continue
        with (root / 'data' / filename).open(encoding='utf-8') as handle:
            data = [[float(row[k]) for k in ['return_pct', 'risk_pct', 'fee_pct',
                                             'threshold_yuan']] for row in csv.DictReader(handle)]
        result['groups'][key] = [solve_target(data, args.budget, t)
                                 for t in (args.target or defaults)]
    target.parent.mkdir(parents=True, exist_ok=True)
    target.write_text(json.dumps(result, ensure_ascii=False, indent=2, allow_nan=False),
                      encoding='utf-8')
    print(json.dumps({k: [{a: x[a] for a in ['target_net_return', 'status',
                                            'minimum_risk'] if a in x} for x in v]
```

```
                    for k, v in result['groups'].items()}, indent=2))


if __name__ == '__main__':
    main()
```

check_target_certificates.py

```
"""Check return targets with exact original fees and independent rational bounds.

For c=1+p and g=r-s-(1+s)p, every original-fee portfolio satisfies
P/M <= s + lambda + R*sum(max(g-lambda*c, 0)/q), lambda >= 0.
A feasible portfolio attaining the corresponding risk lower bound is optimal.
This checker imports neither the numerical model nor an optimization library.
Amounts are continuous; these certificates do not certify rounded orders or markets.
"""
import argparse
import csv
import hashlib
import itertools
import json
from fractions import Fraction
from pathlib import Path
from typing import NamedTuple


BANK = Fraction(1, 20)
ROOT = Path(__file__).resolve().parent
FILES = {'four': 'assets4.csv', 'fifteen': 'assets15.csv'}
TARGETS = {'four': ('1/5', '11/50', '1/4'),
           'fifteen': ('3/10', '33/100', '19/50')}


class Asset(NamedTuple):
    name: str
    r: Fraction
    q: Fraction
    p: Fraction
    u: Fraction


def require(condition, message):
    if not condition:
        raise ValueError(message)


def rational(value):
    """Decimal strings, integers and fractions only; never silently accept NaN."""
    require(not isinstance(value, (bool, float)), 'Exact finite rational input required')
    try:
        return Fraction(value)
    except (TypeError, ValueError, ZeroDivisionError, OverflowError) as error:
        raise ValueError('Exact finite rational input required') from error


def read_assets(path):
```

```
    columns = {'asset', 'return_pct', 'risk_pct', 'fee_pct', 'threshold_yuan'}
    with Path(path).open(encoding='utf-8', newline='') as handle:
        reader = csv.DictReader(handle)
        require(set(reader.fieldnames or ()) == columns, 'Invalid asset CSV columns')
        assets = [Asset(row['asset'], rational(row['return_pct']) / 100,
                        rational(row['risk_pct']) / 100, rational(row['fee_pct']) / 100,
                        rational(row['threshold_yuan'])) for row in reader]
    require(assets and len({a.name for a in assets}) == len(assets),
            'Nonempty distinct asset names required')
    require(all(a.name and a.q > 0 and a.p >= 0 and a.u >= 0 for a in assets),
            'Positive risk and nonnegative fee/threshold required')
    return assets


def dual_coefficient(assets, lam):
    lam = rational(lam)
    require(lam >= 0, 'Nonnegative dual lambda required')
    return sum((max(a.r - BANK - (1 + BANK) * a.p - lam * (1 + a.p), 0) / a.q
                for a in assets), Fraction(0))


def portfolio_values(assets, budget, record):
    """Recompute values before deciding exact or tolerance-qualified feasibility."""
    budget = rational(budget)
    require(budget > 0, 'Positive budget required')
    raw = record['investments_yuan']
    if isinstance(raw, list):
        require(len(raw) == len(assets), 'Wrong investment count')
        raw = dict(zip((a.name for a in assets), raw))
    require(isinstance(raw, dict) and set(raw) == {a.name for a in assets},
            'Every asset purchase must be explicit')
    x = {name: rational(value) for name, value in raw.items()}
    bank = rational(record['bank_yuan'])
    require(all(value >= 0 for value in x.values()), 'Nonnegative purchases required')
    fees = {a.name: a.p * max(x[a.name], a.u) if x[a.name] > 0 else Fraction(0)
            for a in assets}
    spent = bank + sum(x.values()) + sum(fees.values())
    profit = BANK * bank + sum(a.r * x[a.name] for a in assets) - sum(fees.values())
    risk = max(a.q * x[a.name] for a in assets) / budget
    return {'investments_yuan': x, 'fees_yuan': fees, 'bank_yuan': bank,
            'budget_residual_yuan': spent - budget, 'profit_yuan': profit,
            'net_return': profit / budget, 'risk_rate': risk}


def exact_feasible(assets, budget, target, record):
    values = portfolio_values(assets, budget, record)
    require(values['bank_yuan'] >= 0 and values['budget_residual_yuan'] == 0,
            'Original-fee budget/nonnegativity certificate fails')
    require(values['net_return'] >= rational(target), 'Return target certificate fails')
    return values


def verify_minimum(assets, record):
    budget, target = rational(record['budget_yuan']), rational(record['target_net_return'])
    lam, risk = rational(record['lambda']), rational(record['minimum_risk'])
    coefficient = dual_coefficient(assets, lam)
```

```
    require(coefficient > 0 and target > BANK + lam, 'Positive risk lower bound required')
    require(coefficient == rational(record['A']), 'Dual coefficient certificate fails')
    lower = (target - BANK - lam) / coefficient
    values = exact_feasible(assets, budget, target, record)
    require(risk == lower == values['risk_rate'],
            'Feasible risk does not attain the independent lower bound')
    require(all(values['investments_yuan'][a.name] == 0 or a.p == 0 or
                values['investments_yuan'][a.name] >= a.u for a in assets),
            'Proportional-fee equality is outside the original fee regime')
    critical = max((budget * a.u / values['investments_yuan'][a.name]
                    for a in assets if a.p > 0 and values['investments_yuan'][a.name] > 0),
                   default=Fraction(0))
    if critical > 0:
        boundary = {'investments_yuan': {k: v * critical / budget
                                         for k, v in values['investments_yuan'].items()},
                    'bank_yuan': values['bank_yuan'] * critical / budget}
        at_boundary = exact_feasible(assets, critical, target, boundary)
        require(at_boundary['risk_rate'] == risk, 'Funding boundary certificate fails')
    return {'status': 'original_fee_exact_minimum', 'budget_yuan': budget,
            'target_net_return': target, 'minimum_risk': lower,
            'dual': {'lambda': lam, 'A': coefficient}, 'portfolio': values,
            'funding_threshold_yuan': critical, 'funding_boundary_checked': critical > 0}


def verify_maximum(assets, record):
    upper = max(BANK, max((a.r - a.p) / (1 + a.p) for a in assets))
    lam = rational(record['lambda'])
    require(dual_coefficient(assets, lam) == 0 and BANK + lam == upper,
            'Global return upper-bound certificate fails')
    require(rational(record['maximum_return']) == upper, 'Maximum return mismatch')
    values = exact_feasible(assets, record['budget_yuan'], upper, record)
    require(values['net_return'] == upper, 'Global upper bound is not attained')
    return {'budget_yuan': rational(record['budget_yuan']),
            'maximum_return': upper, 'lambda': lam, 'A': Fraction(0),
            'original_fee_attainment': values}


def fixed_fee_support_bounds(assets, budget, risk):
    """Exact upper bounds for every support while all risk caps are below u.

    In each support, allowing zero purchases while still charging its fixed fees
    is optimistic for maximizing its linear objective. Enumerating all supports
    covers every real portfolio. Greedy purchase by r-s solves each such bound.
    """
    budget, risk = rational(budget), rational(risk)
    require(budget > 0 and risk >= 0 and len(assets) <= 8,
            'Positive budget, nonnegative risk and at most eight assets required')
    caps = {a.name: budget * risk / a.q for a in assets}
    require(all(caps[a.name] <= a.u for a in assets), 'Fixed-fee cap regime required')
    rows = []
    for mask in itertools.product((False, True), repeat=len(assets)):
        active = [a for a, used in zip(assets, mask) if used]
        fixed = sum((a.p * a.u for a in active), Fraction(0))
        cash = budget - fixed
        if cash < 0:
            rows.append({'support': [a.name for a in active], 'fixed_fees_yuan': fixed,
```

```
                        'profit_upper_bound_yuan': None})
            continue
        x = {a.name: Fraction(0) for a in assets}
        for a in sorted(active, key=lambda a: a.r - BANK, reverse=True):
            if a.r > BANK:
                x[a.name] = min(cash, caps[a.name])
                cash -= x[a.name]
        profit = BANK * cash + sum(a.r * x[a.name] for a in assets) - fixed
        rows.append({'support': [a.name for a in active], 'fixed_fees_yuan': fixed,
                     'profit_upper_bound_yuan': profit, 'investments_yuan': x,
                     'bank_yuan': cash,
                     'support_attained': all(x[a.name] > 0 for a in active)})
    return rows


def verify_small_budget(assets, record):
    budget, target = rational(record['budget_yuan']), rational(record['target_net_return'])
    risk, lam = rational(record['relaxed_risk']), rational(record['lambda'])
    require(budget == 100 and target == Fraction(1, 5) and len(assets) == 4,
            'Expected the four-asset 100-yuan counterexample')
    coefficient = dual_coefficient(assets, lam)
    require(coefficient > 0 and risk == (target - BANK - lam) / coefficient,
            'Relaxation lower-bound certificate fails')
    rows = fixed_fee_support_bounds(assets, budget, risk)
    best = max((r for r in rows if r['profit_upper_bound_yuan'] is not None),
               key=lambda r: r['profit_upper_bound_yuan'])
    require(best['support_attained'], 'Best support bound has no attainment certificate')
    attained = exact_feasible(assets, budget, best['profit_upper_bound_yuan'] / budget, best)
    maximum = attained['profit_yuan']
    require(maximum == best['profit_upper_bound_yuan'] ==
            rational(record['maximum_profit_at_relaxed_risk_yuan']),
            'Fixed-fee support maximum certificate fails')
    require(maximum < budget * target, 'Counterexample does not exclude the target')
    examples = [exact_feasible(assets, budget, target, record[k])
                for k in ('feasible_only', 'strictly_better_feasible')]
    for values, key in zip(examples, ('feasible_only', 'strictly_better_feasible')):
        require(values['risk_rate'] == rational(record[key]['risk_rate']),
                'Feasible example risk mismatch')
    require(risk < examples[1]['risk_rate'] < examples[0]['risk_rate'],
            'Strictly better feasible example certificate fails')
    return {'budget_yuan': budget, 'target_net_return': target, 'relaxed_risk': risk,
            'support_count': len(rows), 'all_support_upper_bounds': rows,
            'maximum_profit_at_relaxed_risk_yuan': maximum,
            'strict_target_shortfall_yuan': budget * target - maximum,
            'no_feasible_portfolio_at_or_below_relaxed_risk': True,
            'feasible_only': {**examples[0], 'not_minimum_verified': True},
            'strictly_better_feasible': {**examples[1], 'minimum_verified': False},
            'exact_small_budget_minimum_claimed': False}


def verify_bundle(reference, data_root=ROOT):
    require(reference['format'] == 'cumcm-return-target-certificates-v1' and
            rational(reference['bank_rate']) == BANK and
            reference['risk_definition'] == 'max_i(q_i*x_i)/initial_budget' and
            reference['money_unit'] == 'yuan' and
            set(reference['groups']) == set(FILES), 'Invalid certificate bundle')
```

```
    groups, inputs, assets_by_group = {}, {}, {}
    for name, filename in FILES.items():
        group = reference['groups'][name]
        require(group['csv'] == 'data/' + filename, 'Unexpected asset CSV path')
        path = Path(data_root) / 'data' / filename
        digest = hashlib.sha256(path.read_bytes()).hexdigest()
        require(digest == group['csv_sha256'], 'Asset CSV hash mismatch')
        assets = read_assets(path)
        require(len(assets) == (4 if name == 'four' else 15), 'Wrong asset count')
        cases = group['cases']
        require(len(cases) == 3 and {rational(c['target_net_return']) for c in cases} ==
                {Fraction(t) for t in TARGETS[name]} and
                all(rational(c['budget_yuan']) == 1_000_000 for c in cases),
                'The six million-yuan target cases must be complete')
        checked = [verify_minimum(assets, c) for c in cases]
        maximum = verify_maximum(assets, group['maximum'])
        bank = []
        for target in (Fraction(0), Fraction(3, 100), BANK):
            values = exact_feasible(assets, 1_000_000, target,
                                    {'investments_yuan': {a.name: 0 for a in assets},
                                     'bank_yuan': 1_000_000})
            bank.append({'target_net_return': target, 'minimum_risk': values['risk_rate'],
                         'status': 'bank_only_exact_minimum'})
        above = [rational(t) for t in group['unreachable_targets']]
        require(above, 'Unreachable target examples required')
        above.append(maximum['maximum_return'] + Fraction(1, 1_000_000))
        require(all(t > maximum['maximum_return'] for t in above),
                'An unreachable target does not exceed the global bound')
        groups[name] = {'target_cases': checked, 'maximum': maximum, 'bank_boundaries': bank,
                        'unreachable_targets': [{'target_net_return': t, 'status': 'infeasible',
                                                 'evidence': 'strict global return upper bound'}
                                                for t in above]}
        inputs[name] = {'csv': group['csv'], 'sha256': digest}
        assets_by_group[name] = assets
    small = reference['small_budget_counterexample']
    require(small['group'] == 'four', 'Small-budget group must be four')
    return {'method': 'stdlib Fraction: original-fee primal, proportional dual, full fixed-fee
    support enumeration',
            'risk_definition': 'max_i(q_i*x_i)/initial_budget',
            'preference_source': 'illustrative input targets, not company preferences',
            'source_csv': inputs, 'groups': groups,
            'small_budget_counterexample': verify_small_budget(assets_by_group['four'], small)},
            assets_by_group


def verify_numeric(result, assets_by_group, exact, tolerance=Fraction(1, 100_000_000)):
    """Check numerical values with a stated rate/relative-money tolerance.

    An arbitrary feasible solver output receives no optimality verdict. Exact
    optimality is borrowed only from an already checked same-budget/target bound.
    """
    require(set(result['groups']) <= set(FILES) and result['groups'], 'Invalid numerical groups')
    checked = []
    for name, rows in result['groups'].items():
        assets = assets_by_group[name]
        upper = exact['groups'][name]['maximum']['maximum_return']
```

```
        known = {(c['budget_yuan'], c['target_net_return']): c['minimum_risk']
                 for c in exact['groups'][name]['target_cases']}
        require(rows, 'Empty numerical group')
        seen = set()
        for row in rows:
            budget, target = rational(row['budget_yuan']), rational(row['target_net_return'])
            require(budget > 0 and (budget, target) not in seen, 'Invalid or duplicate numerical
            case')
            seen.add((budget, target))
            require(abs(rational(row['proportional_fee_return_upper_bound']) - upper) <=
            tolerance,
                    'Numerical global return bound mismatch')
            if row['status'] == 'infeasible':
                require(target > upper, 'No independent certificate for numerical infeasibility')
                checked.append({'group': name, 'budget_yuan': budget, 'target_net_return': target,
                                'status': 'infeasible_by_exact_global_bound'})
                continue
            require(row['status'] == 'optimal', 'Unsupported numerical solver status')
            values = portfolio_values(assets, budget, row)
            money_tolerance = budget * tolerance
            require(values['bank_yuan'] >= -money_tolerance and
                    abs(values['budget_residual_yuan']) <= money_tolerance,
                    'Numerical original-fee budget fails')
            require(len(row['fees_yuan']) == len(assets) and
                    all(abs(rational(f) - values['fees_yuan'][a.name]) <= money_tolerance
                        for a, f in zip(assets, row['fees_yuan'])), 'Numerical original fee
                        mismatch')
            require(abs(rational(row['net_return']) - values['net_return']) <= tolerance and
                    values['net_return'] >= target - tolerance, 'Numerical return fails')
            risk = rational(row['minimum_risk'])
            require(abs(risk - values['risk_rate']) <= tolerance, 'Numerical risk mismatch')
            bound = Fraction(0) if target <= BANK else known.get((budget, target))
            if bound is not None:
                require(abs(risk - bound) <= tolerance, 'Numerical risk differs from exact
                minimum')
            checked.append({'group': name, 'budget_yuan': budget, 'target_net_return': target,
                            'status': 'matches_exact_minimum_with_tolerance' if bound is not None
                            else 'feasible_with_tolerance_optimality_unverified',
                            'exact_minimum_verified': bound is not None,
                            'rate_and_relative_money_tolerance': tolerance,
                            'recomputed_original_fee_values': values})
    return checked


def encode(value):
    if isinstance(value, Fraction):
        return str(value)
    if isinstance(value, dict):
        return {k: encode(v) for k, v in value.items()}
    if isinstance(value, list):
        return [encode(v) for v in value]
    return value


def read_json(path):
    def reject_constant(value):
```

```
        raise ValueError('Nonfinite JSON number: ' + value)
    return json.loads(Path(path).read_text(encoding='utf-8'),
                      parse_float=Fraction, parse_constant=reject_constant)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--reference', type=Path, default=ROOT / 'reference/target-certificates.json')
    parser.add_argument('--numeric', type=Path, help='Optional numerical return-targets.json to check')
    parser.add_argument('--output', type=Path, default=ROOT / 'reproduced/target-checks.json')
    args = parser.parse_args()
    if args.output.exists():
        raise FileExistsError('Refusing to replace existing target checks')
    checked, assets = verify_bundle(read_json(args.reference))
    checked['certificate_sha256'] = hashlib.sha256(args.reference.read_bytes()).hexdigest()
    if args.numeric:
        checked['numerical_checks'] = verify_numeric(read_json(args.numeric), assets, checked)
        checked['numerical_source_sha256'] = hashlib.sha256(args.numeric.read_bytes()).hexdigest()
    args.output.parent.mkdir(parents=True, exist_ok=True)
    with args.output.open('x', encoding='utf-8') as handle:
        json.dump(encode(checked), handle, ensure_ascii=False, indent=2, allow_nan=False)
        handle.write('\n')
    print(json.dumps({'exact_minimum_cases': 6, 'small_budget_supports': 16,
                      'numerical_cases': len(checked.get('numerical_checks', [])), 'passed':
                      True}))


if __name__ == '__main__':
    main()
```